YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT A CONSTITUENT UNIT OF YENEPOYA (DEEMED TO BE UNIVERSITY) BALMATTA, MANGALORE

LOW LEVEL DESIGN

ON

NATURAL LANGUAGE INTERFACE FOR HEALTHCARE INFRASTRUCTURE

SUBMITTED BY

MUHAMMED MISHAL P

REG NO :

23BCAICD075

LH-06

GUIDED BY

MISS SINDHU

SUBMISSION DATE

17-04-2026

Table of Contents

1 Introduction
- 1.1 Scope of the document
- 1.2 Intended Audience
- 1.3 System overview

2 System Design
- 2.1 Application Design
- 2.2 Process Flow
- 2.3 Information Flow
- 2.4 Components Design
- 2.5 Key Design Considerations
- 2.6 API Catalogue

3 Data Design
- 3.1 Data Model
- 3.2 Data Access Mechanism
- 3.3 Data Retention Policies
- 3.4 Data Migration

4 Interfaces

5 State and Session Management

6 Caching

7 Non-Functional Requirements
- 7.1 Security Aspects
- 7.2 Performance Aspects

8 References

1. Introduction

1.1 Scope of the Document

This document describes the detailed internal design of the AI-based healthcare chatbot system. It explains how different modules are implemented, how data flows through the system, and how user queries are processed to generate meaningful and safe responses.

1.2 Intended Audience

- Project evaluators
- Development team
- Academic supervisors
- Anyone interested in understanding system implementation

1.3 System Overview

The system is a web-based chatbot application that allows users to interact using natural language. The chatbot processes user input using a transformer-based NLP model and provides responses related to symptoms, medicines, diet, lifestyle, mental health, and first aid.
In addition to AI-based responses, the system also uses rule-based checks to handle greetings and critical health situations, ensuring safety and reliability.

2. System Design

2.1 Application Design

The system is designed using a modular layered architecture:

- **Frontend Layer**
  Provides a chat interface using HTML, CSS, and JavaScript. It captures user input and displays responses dynamically.
- **Backend Layer (Flask)**
  Handles HTTP requests and routes them to appropriate modules. It acts as a bridge between the user interface and AI logic.
- **AI Processing Layer**
  Uses a pre-trained Sentence Transformer model (all-MiniLM-L6-v2) to understand the semantic meaning of user queries.
- **Rule-Based Layer**
  Handles simple queries such as greetings and identifies critical conditions where safe responses are required.
- **Knowledge Layer**
  Stores predefined healthcare-related question-answer pairs used for response matching.
- **Logging Layer**
  Records user interactions including queries and responses for tracking and analysis.

2.2 Process Flow

User enters query →
Frontend sends request →
Backend receives input →
Input validation is performed →
Rule-based checks are applied →
If not matched, AI engine processes input →
Input converted into embeddings →
Cosine similarity is computed →
Best matching response is selected →
If similarity is low, fallback response is given →
Response sent back to frontend →
Interaction stored in log file

2.3 Information Flow

- User input flows from browser to Flask backend
- Backend forwards input to AI engine
- AI engine converts text into embeddings
- Similarity matching is performed with stored data
- Best response is selected
- Response returned to frontend
- Logging module stores interaction

2.4 Components Design

- **User Interface Module**
    - Accepts user input through chat
    - Displays responses instantly
    - Supports Enter key for sending messages
- **Server Module (Flask)**
    - Handles routing (/, /chat)
    - Manages request-response cycle
    - Validates incoming data
- **AI Engine Module**
    - Loads Sentence Transformer model
    - Converts input into embeddings

    - Uses cosine similarity for matching
    - Selects best response based on similarity score
    - Applies threshold to avoid incorrect responses
- **Rule-Based Module**
    - Detects greetings like “hi”, “hello”
    - Identifies critical situations (e.g., severe symptoms)
    - Overrides AI output when safety is required
- **Knowledge Base Module**
    - Stores structured Q&A pairs
    - Covers multiple healthcare categories
    - Acts as source for response generation
- **Logging Module**
    - Stores user queries and responses
    - Maintains timestamps
    - Saves data in logs/chat_log.txt

2.5 Key Design Considerations

- Ensure safe and non-prescriptive healthcare responses
- Maintain fast response time using lightweight model
- Improve accuracy using semantic similarity
- Keep system modular for easy updates
- Provide fallback for unknown queries

2.6 API Catalogue

2.6 API Catalogue

Endpoint	Method	Purpose	Description
/	GET	Loads chatbot interface	Returns the index.html page and initializes the chat UI
/chat	POST	Processes user query	Accepts user input in JSON format, sends it to AI engine performs rule-based

			checks and NLP processing, and returns the generated response in JSON format

3. Data Design

3.1 Data Model

The system uses an in-memory dataset consisting of structured question-answer pairs.
Each entry includes:

- Query pattern
- Corresponding response

Data is categorized into symptoms, medicines, diet, lifestyle, mental health, and first aid.

3.2 Data Access Mechanism

- Data is stored as Python data structures
- NLP model converts input into embeddings
- Cosine similarity is used to find best match
- Response is selected based on similarity score

3.3 Data Retention Policy

- User interactions are stored in log files
- Logs are saved in logs/chat_log.txt
- Each entry contains timestamp, query, and response

3.4 Data Migration

The current system uses file-based storage where all chatbot data and logs are maintained locally.
User interactions are stored in a text file (chat_log.txt), and the knowledge base is maintained in in-memory data structures.

For future scalability, the system can be extended to use database solutions such as SQLite, MySQL, or MongoDB.
This will enable efficient data management, faster retrieval, and support for larger datasets.

Additionally, database integration will allow features such as:

- Persistent chat history
- User-specific data storage

- Analytics and reporting

4. Interfaces

- **Web-based Chat Interface**
  Provides an interactive user interface where users can type queries and receive responses in real time. The interface supports dynamic message display and user-friendly interaction.
- **Backend REST API**
  Handles communication between frontend and backend.
  The API receives user input, processes it through the AI engine, and returns the generated response in JSON format.

5. State and Session Management

The system follows a stateless architecture, where each user request is processed independently without maintaining session information.

This approach simplifies system design and improves scalability.
However, chat history is stored separately in log files for tracking and analysis purposes.

In future versions, session management can be introduced to:

- Maintain conversation context
- Provide personalized responses
- Improve user experience

6. Caching

Caching is not explicitly implemented in the current system.
However, to improve performance, the NLP model (Sentence Transformer) is loaded once during application startup and reused for all incoming requests.

This avoids repeated model loading and significantly reduces response time.

In future enhancements, caching techniques can be introduced to:

- Store frequently asked queries and responses
- Reduce repeated computation for similar inputs
- Improve overall system efficiency and scalability

7. Non-Functional Requirements

7.1 Security Aspects

- Input Validation
  Ensures that user inputs are properly handled to prevent invalid or unexpected data from affecting system behavior.
- Safe Response Generation
  The chatbot avoids providing medical prescriptions or risky advice and limits responses to general healthcare guidance.
- Handling Critical Conditions
  The system identifies potentially serious health-related queries and responds with safe suggestions, such as consulting a doctor.

7.2 Performance Aspects

- **Fast Response Time**
  The use of a lightweight pre-trained model ensures quick processing of user queries.
- **Efficient Similarity Computation**
  Cosine similarity is used to quickly compare user input with stored data and retrieve the best response.
- **Minimal Processing Delay**
  The system processes requests in real time with minimal latency, providing a smooth user experience.
- **Optimized Resource Usage**
  The model is reused across requests, reducing memory and computation overhead.

8. References

- Python Documentation
- Flask Documentation
- Sentence Transformers Documentation
- NLP Concepts